سیستم ETL پایش شبکه – گزارش کار جامع

ETLpipline از CSV به Apache Cassandra

۱. معرفی پروژه

این پروژه یک خط لوله ETL (استخراج، تبدیل، بارگذاری) برای داده‌های پایش شبکه است. فایل CSV شامل متریک‌های جمع‌آوری‌شده از منابع شبکه (روتر، سوئیچ، فایروال و غیره) را می‌خواند، در پایتون اعتبارسنجی و تبدیل می‌کند و در Apache Cassandra ذخیره می‌کند. داده‌ها طوری ذخیره می‌شوند که دو پرسش زیر به‌صورت کارآمد پاسخ داده شوند:

- **Q1:** یک متریک مشخص از یک منبع مشخص، در یک بازه زمانی مشخص چه مقادیری داشته است؟
- **Q2:** یک منبع مشخص چه متریک‌هایی دارد؟

نسخه اول پروژه یک نمونه اولیه کارا بود، اما چند مشکل طراحی و پیاده‌سازی داشت. این گزارش مشکلات، مدل داده بازطراحی‌شده و کد اصلاح‌شده را شرح می‌دهد.

۲. اهداف

- خواندن متریک‌های شبکه از CSV و اعتبارسنجی ستون‌های الزامی.
- تبدیل مقادیر خام CSV به نوع‌های درست Cassandra (زمان و مقدار عددی).
- مدل‌سازی داده بر اساس دو کوئری Q۱ و Q۲ با اندازه پارتیشن کنترل‌شده.
- نگهداری داده به مدت یک سال با TTL در Cassandra.
- نوشتن همزمان و ایمن در همه جداول.
- فراهم کردن پایه‌ای برای داشبورد، هشدار و تحلیل سری زمانی.

۳. معماری

فایل CSV ← پایتون (استخراج و تبدیل) ← درایور Cassandra (نوشتن همزمان) ← دو جدول Cassandra

اسکریپت روی ماشین میزبان اجرا می‌شود و از طریق پورت ۹۰۴۲ (که از کانتینر Docker منتشر شده) به Cassandra وصل می‌شود. آدرس اتصال دیگر ثابت نیست و از تنظیمات یا متغیر محیطی `CASSANDRA_HOSTS` خوانده می‌شود؛ مقدار پیش‌فرض `127.0.0.1` است.

۴. فناوری‌ها

کاربرد	فناوری
منطق ETL	Python 3
خواندن و تبدیل CSV	pandas
اتصال، prepared statement و اجرای همزمان	cassandra-driver
ذخیره‌سازی توزیع‌شده	Apache Cassandra 5.0
اجرای Cassandra در کانتینر	Docker

۵. مدل داده

مدل با رویکرد «اول کوئری» طراحی شد: ابتدا `WHERE` هر کوئری نوشته شد و سپس برای آن جدول طراحی شد.

۵.۱ کوئری Q۱ – مقادیر یک متریک در بازه زمانی

```
SELECT collected_at, value FROM metrics
WHERE resource = ? AND metric_name = ? AND time_bucket = ?
  AND collected_at >= ? AND collected_at < ?;
```

۵.۲ کوئری Q۲ – متریک‌های یک منبع

```
SELECT metric_name FROM metrics_metrics_by_resource
WHERE resource = ?;
```

۵.۳ schema

```
CREATE KEYSPACE IF NOT EXISTS network_monitoring
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

CREATE TABLE IF NOT EXISTS network_monitoring.metrics (
  resource     text,
  metric_name  text,
  time_bucket  date,
  collected_at timestamp,
  value        double,
  PRIMARY KEY ((resource, metric_name, time_bucket), collected_at)
) WITH CLUSTERING ORDER BY (collected_at DESC);

CREATE TABLE IF NOT EXISTS network_monitoring.metrics_metrics_by_resource (
  resource    text,
  metric_name text,
  PRIMARY KEY ((resource), metric_name)
);
```

نام جدول دوم برابر است با نام جدول تنظیم‌شده به‌علاوه پسوند `_metrics_by_resource`.

۶. قالب داده CSV

سرستون‌های الزامی (با هر ترتیبی): `value`، `collected_at`، `metric_name`، `resource`.

ستون	معنی	مثال
resource	دستگاه پایش‌شده	Router-01
metric_name	نام اندازه‌گیری	CPU Usage
collected_at	زمان به شکل YYYY-MM-DD HH:MM (UTC) یا Unix seconds	2018-12-16 22:25
value	مقدار عددی متریک	72

۷. ساختار کد

کد `ETL.py` در سه کلاس سازمان‌دهی شده است.

۷.۱ کلاس `Validation`

شامل ثابت‌ها و توابع کمکی اعتبارسنجی (همه static):

- ثابت‌ها: `TIME_FORMAT`، `IDENTIFIER_RE`، `DEFAULT_TTL`، `REQUIRED_COLUMNS` (برابر `%Y-%m-%d %H:%M%`).
- `validate_identifier(name, kind)`: نام keyspace و table . را با عبارت منظم فهرست سفید می‌کند. از طریق نام‌ها جلوگیری می‌کند injection کرد، پس این کار از bind را نمی‌شود به‌صورت پارامتر CQL شناسه‌های.
- `parse_cli_time(text)`: رشته زمان واردشده توسط کاربر را تبدیل می‌کند.
- `get_hosts(arg_hosts)`: آدرس‌های اتصال را از آرگومان، متغیر محیطی `CASSANDRA_HOSTS` یا مقدار برمی‌گرداند `127.0.0.1`.

۷.۲ کلاس `ETL`

- `read_csv(file_path)` – **استخراج.** CSV را می‌خواند و اگر ستون الزامی کم باشد، خطایی با نام ستون‌های گم‌شده می‌دهد.
- `transform(df)` – **تبدیل.** `collected_at` را به timestamp با وقت UTC تبدیل می‌کند)هم رشته تاریخ-زمان و تبدیل می‌کند)مقدار نامعتبر به‌جای پنهان شدن، خطا float را به `value`، (عددی را می‌پذیرد Unix seconds هم روزانه را می‌سازد `time_bucket` می‌دهد(، فاصله‌های اضافی ستون‌های متنی را حذف می‌کند، سلول خالی را رد می‌کند و.
- `create_schema(session, table)` – هر دو جدول را در صورت نبودن می‌سازد.
- `load(session, df, table, ttl)` – دو دستور **بارگذاری.** `INSERT ... USING TTL ?` را prepare می‌کند و همه ردیف‌ها را در جدول اصلی و جفت‌های یکتای `(resource, metric_name)` را در جدول جستجو برابر ۱۰۰ concurrency و `execute_concurrent_with_args` می‌نویسد؛ با.

۷.۳ کلاس `CQL`

- `q1(session, table, resource, metric, start, end)` – bucketهای روزانه پوشش‌دهنده `[start, end)` را حساب می‌کند، همزمان کوئری می‌زند و ردیف‌ها را مرتب‌شده برمی‌گرداند.
- `q2(session, table, resource)` – همه نام متریک‌های یک منبع را از جدول جستجو برمی‌گرداند.

۷.۴ نقطه ورود برنامه

جریان اجرا زیر `if __name__ == "__main__":` است. نام keyspace و table را اعتبارسنجی می‌کند، به Cassandra وصل می‌شود، دستور درخواستی (`q1`، `load` یا `q2`) را اجرا می‌کند و همیشه در بلوک `finally` عبارت `cluster.shutdown()` را صدا می‌زند. خطاهای مورد انتظار (`OSError`، `RuntimeError`، `ValueError`) روی خروجی خطای استاندارد با کد خروج ۱ چاپ می‌شوند.

۸. پارامترهای پیکربندی

تجزیه آرگومان‌های خط فرمان عمداً حذف شد. پارامترهایی که قبلاً از parser می‌آمدند قرار است از فایل `conf.yaml` تأمین شوند:

پارامتر	مصرف‌کننده	توضیح
hosts	همه	پیش‌فرض: `$CASSANDRA_HOSTS` یا `127.0.0.1`
command	همه	`q2` یا `q1`، `load`

پارامتر	مصرف‌کننده	توضیح
keyspace	همه	با الگوی شناسه اعتبارسنجی می‌شود
table	همه	با الگوی شناسه اعتبارسنجی می‌شود
csv_file	load	مسیر فایل CSV ورودی
ttl	load	بر حسب ثانیه؛ پیش‌فرض ۳۱,۵۳۶,۰۰۰
create_schema	load	true / false
resource	q1 و q2	نام منبع
metric	q1	نام متریک
start	q1	شامل خود زمان، UTC، `YYYY-MM-DD HH:MM`
end	q1	بدون خود زمان، UTC، `YYYY-MM-DD HH:MM`

۹. راه‌اندازی و اجرا

۹.۱ نصب وابستگی‌ها (Ubuntu/Debian)

```
sudo apt update
sudo apt install python3 python3-venv python3-pip
cd [your user/file disten]
python3 -m venv venv
source venv/bin/activate
python -m pip install pandas cassandra-driver
```

برای فایل پیکربندی، یک پارسر YAML مثل PyYAML هم لازم خواهد بود.

۹.۲ اجرای Cassandra با Docker

```
docker run --name cassandra-node -p 9042:9042 -d cassandra:5.0
docker ps
docker logs cassandra-node
docker exec -it cassandra-node cqlsh
```

اگر کانتینر از قبل وجود دارد، با `docker start cassandra-node` آن را اجرا کنید. برای داده‌های مهم از ذخیره‌سازی پایدار استفاده کنید.

۹.۳ ساخت keyspace و جداول

دستورهای بخش ۶.۳ را در `cqlsh` اجرا کنید، یا گزینه `create_schema` را فعال کنید تا اسکریپت جداول را بسازد (خود keyspace باید از قبل وجود داشته باشد، چون اسکریپت به آن وصل می‌شود).

۹.۴ بررسی داده‌ها

```
SELECT * FROM network_monitoring.metrics LIMIT 20;
SELECT metric_name FROM network_monitoring.metrics_metrics_by_resource WHERE resource
= 'Router-01';
SELECT collected_at, value FROM network_monitoring.metrics
WHERE resource = 'Router-01' AND metric_name = 'CPU Usage'
```

```
  AND time_bucket = '2018-12-16'
  AND collected_at >= '2018-12-16 00:00' AND collected_at < '2018-12-17 00:00';
```

نتایج را با CSV ورودی مقایسه کنید. تعداد ردیف‌های گزارش‌شده توسط loader باید با تعداد ردیف‌های CSV و تعداد جفت‌های یکتای (resource, metric_name) برابر باشد.

۱۰. چک‌لیست عملیاتی

- کانتینر Cassandra اجراست و پورت ۹۰۴۲ منتشر شده است.
- keyspace و هر دو جدول وجود دارند.
- محیط پایتون pandas و cassandra-driver را دارد.
- سرستون‌های CSV شامل collected_at، metric_name، resource و value هستند.
- collected_at به شکل YYYY-MM-DD HH:MM (UTC) یا Unix seconds است و value عددی است.
- پیکربندی دستور، table، keyspace و پارامترهای همان دستور را فراهم می‌کند.
- کوئری‌های SELECT ردیف‌های ذخیره‌شده را تأیید می‌کنند.

۱۲. نیازمندی‌های اجرا و توضیح فایل conf.yaml

نیاز دارید. این ETL.pyکنار conf.yamlدر دسترس و فایل Cassandra به پایتون، سه بسته، یک ETL برای اجرای بخش نیازمندی‌ها و معنی هر کلید فایل پیکربندی را توضیح می‌دهد.

۱۲.۱ نیازمندی‌های اجرا

- **Python 3**همراه با venv) ترجیحاً در یک محیط مجازی جدا.(
- **بسته‌های پایتون**pandas:، cassandra-driverو pyyaml) برای خواندن conf.yaml.(
- **Apache Cassandra 5.0**در حال اجرا و در دسترس روی پورت ۹۰۴۲) مثلاً با.(Docker
- **keyspace**از قبل ساخته‌شده؛ مثلاً network_monitoring.ساخت جداول را می‌توان به create_schema: truesپرد.
- **فایل CSV** با سرستون‌های resource، metric_name، collected_at، value؛ زمان به شکل YYYY-MM-DD HH:MM (UTC)یا Unix seconds و مقدار عددی.
- **فایل‌های ETL.py و conf.yaml**در یک پوشه، و دسترسی شبکه از ماشین اجراکننده به.Cassandra

```
python -m pip install pandas cassandra-driver pyyaml
```

۱۲.۲ توضیح فایلconf.yaml

پارامترهایی که پیش‌تر از خط فرمان می‌آمدند، اکنون در یک فایل YAML سه‌بخشی جمع شده‌اند: بخش مشترک، بخش loadو بخش q1/q2.از هر دستور فقط کلیدهای مربوط به همان دستور استفاده می‌شود.

کلید	استفاده در	توضیح	مقدار نمونه
hosts	همه	آدرس نودهای Cassandra، جداشده با ویرگول. اگر خالی 127.0.0.1و سپس $CASSANDRA_HOSTSباشد،	"127.0.0.1"
command	همه	دستور اجرا: load یا q1 یا q2	load
keyspace	همه	نام keyspaceمی‌شود؛ با الگوی شناسه اعتبارسنجی	network_monitoring
table	همه	نام جدول اصلی؛ جدول دوم با پسوند _metrics_by_resource ساخته می‌شود	metrics

csv_file	load	مسیر فایل CSV ورودی	network_metrics.csv
ttl	load	مدت نگهداری هر ردیف بر حسب ثانیه؛ پیش‌فرض یک سال	31536000
create_schema	load	اگر true باشد، جداول در صورت نبود ساخته می‌شوند	false
resource	q1 و q2	نام منبع مورد پرسش	Router-01
metric	q1	نام متریک مورد پرسش	CPU Usage
start	q1	آغاز بازه، شامل خود زمان، UTC	"2018-12-16 00:00"
end	q1	پایان بازه، بدون خود زمان، UTC	"2018-12-23 00:00"

```
hosts: "127.0.0.1"
command: load               # load | q1 | q2
keyspace: network_monitoring
table: metrics

csv_file: network_metrics.csv
ttl: 31536000
create_schema: false

resource: Router-01
metric: CPU Usage
start: "2018-12-16 00:00"
end: "2018-12-23 00:00"
```

۱۳. پیوست – مرجع سریع Vim

کلیدها	کار
`vim ETL.py`	باز کردن فایل
`i`	حالت Insert
Enter و `w:` سپس ،`Esc`	ذخیره
Enter و `wq:` سپس ،`Esc`	ذخیره و خروج
`% python3!:` سپس ،`Esc`	اجرای فایل جاری
`python -m py_compile ETL.py`	بررسی سینتکس

۱۴. جمع‌بندی

خط لوله از یک نمونه اولیه تک‌جدولی به یک مدل Cassandra مبتنی بر کوئری بازطراحی شد: یک جدول برای خواندن بازه زمانی (Q۱) با bucket روزانه، و یک جدول جستجو برای فهرست متریک‌های هر منبع (Q۲). کد اکنون داده واقعی پروژه (رشته‌های تاریخ-زمان و مقادیر عددی) را به نوع‌های درست Cassandra تبدیل می‌کند، TTL یک‌ساله اعمال می‌کند، در هر دو جدول همزمان می‌نویسد، شناسه‌ها را اعتبارسنجی می‌کند و همیشه اتصال کلاستر را می‌بندد. کارهای باقی‌مانده: فایل پیکربندی، تست روی کلاستر واقعی و ثبت خروجی واقعی کوئری‌ها در گزارش.